

Python & SQL Study Notes

Data Cleaning in Python • Correlated Subqueries in SQL

1. Python — Employee Data Cleaning

Problem

- Extra spaces in name/department fields
- Empty names or departments
- Invalid salary values such as "invalid"

Final Solution

```
def clean_employees(employees):
    cleaned = []

    for employee in employees:

        employee["name"] = employee["name"].strip()
        employee["department"] = employee["department"].strip()

        if employee["name"] == "" or employee["department"] == "":
            continue

        try:
            employee["salary"] = float(employee["salary"])
        except ValueError:
            continue

        cleaned.append(employee)

    return cleaned
```

Important Concepts

.strip()

```
employee["name"].strip()
```

Removes leading/trailing whitespace. Because strings are immutable, the result must be re-assigned back: `employee["name"] = employee["name"].strip()`

continue vs break

- `continue` → skip the current employee and move to the next one
- `break` → stop the entire loop

For data cleaning, the pattern below is usually what's needed:

```
if invalid:
    continue
```

try / except

```
try:
    employee["salary"] = float(employee["salary"])
except ValueError:
    continue
```

`float("75000")` works, but `float("invalid")` raises a `ValueError` — so that employee is skipped instead of crashing the program.

Python Pattern Learned

```
Raw data
↓
Clean
↓
Validate
↓
Convert
↓
Keep valid records
```

This pipeline is directly useful for ML / Data Engineering work, where data preprocessing is common.

2. SQL — Highest Salary Per Department

LeetCode 184 — Department Highest Salary (solved without JOIN)

Step 1 — Maximum Salary Per Department

```
SELECT department, MAX(salary)
FROM Employees
GROUP BY department;
```

This gives the maximum salary for every department, but not which employee holds that salary.

Step 2 — Correlated Subquery

```
WHERE e.salary = (
  SELECT MAX(e2.salary)
  FROM Employee e2
  WHERE e2.departmentId = e.departmentId
)
```

Meaning: for every employee e, find the maximum salary among employees e2 in the same department, then check:

```
employee salary == department maximum?
↓
YES → return employee
NO → discard employee
```

3. Correlated Subquery — Core Concept

Two references to the same table are used: FROM Employee e and FROM Employee e2.

- e → current employee (outer query)
- e2 → employees being searched (inner query)

The key connection is `e2.departmentId = e.departmentId` — the inner query depends on the current row of the outer query. That dependency is exactly why it's called a **correlated subquery**.

4. Getting Department Name Without JOIN

Since LeetCode 184 has a separate Department table, another subquery maps the ID to a name:

```
(
  SELECT e3.name
  FROM Department e3
  WHERE e3.id = e.departmentId
)
```

```
Employee.departmentId  
↓  
Department.id  
↓  
Department.name
```

Final LeetCode 184 Query — No JOIN

```
SELECT  
(  
  SELECT e3.name  
  FROM Department e3  
  WHERE e3.id = e.departmentId  
) AS Department,  
e.name AS Employee,  
e.salary AS Salary  
FROM Employee e  
WHERE e.salary = (  
  SELECT MAX(e2.salary)  
  FROM Employee e2  
  WHERE e2.departmentId = e.departmentId  
);
```

Note

One typo occurred during the attempt: `e . departmenId` instead of `e . departmentId`. The overall logic was correct.

What Was Learned Today

Python

- Dictionary-based data processing
- `.strip()` for whitespace removal
- `continue` for loop control
- `try/except` and `ValueError` handling
- String → float conversion
- Building a data cleaning pipeline
- Constructing a cleaned output list

SQL

- Correlated subqueries
- Table aliases
- `MAX()` aggregate function
- Comparing outer vs. inner query values
- Highest salary per group
- Using a subquery instead of `JOIN`
- Mapping `departmentId` → `Department.id` → name
- Handling ties automatically